UNIT 2: SOLVING PROBLEMS BY SEARCHING

Comprehensive Examination Study Notes & Problem-Solving Manual

TABLE OF CONTENTS

1.0 Problem-Solving Agents & Problem Formulation | 2.0 Uninformed Search Strategies (BFS, DFS, UCS, DLS, IDS)
3.0 Informed (Heuristic) Search (GBFS, A*, Heuristics) | 4.0 Local Search Algorithms (Hill Climbing, GA)
5.0 Adversarial Search & Game Playing (Minimax, Alpha-Beta) | 6.0 Exam Review, Solved PYQ Bank & Syllabus Checklist

1.0 PROBLEM-SOLVING AGENTS & PROBLEM FORMULATION

A **Problem-Solving Agent** is a goal-based agent that decides what to do by finding sequences of actions that lead to desirable states. When the environment is **deterministic, fully observable, static, and discrete**, the agent can formulate a clean goal, construct an explicit problem formulation, run an offline search algorithm to compute a complete solution sequence, and execute the plan open-loop without sensing at every step.

1.1 The Four Operational Phases of Problem Solving

• **Goal Formulation:** The agent specifies its target objective based on performance measures and current state (e.g., reaching Bucharest or solving the 8-puzzle).

• **Problem Formulation:** The agent decides what abstract states and actions to consider in order to reach the goal, ignoring irrelevant real-world details.

• **Search:** The agent simulates action sequences through a state-space model without executing physical actions, finding a path from initial to goal state.

• **Execution:** The agent executes the sequence of actions one by one in the real physical environment.

Figure 2.1: Problem-Solving Agent Operational Sequence

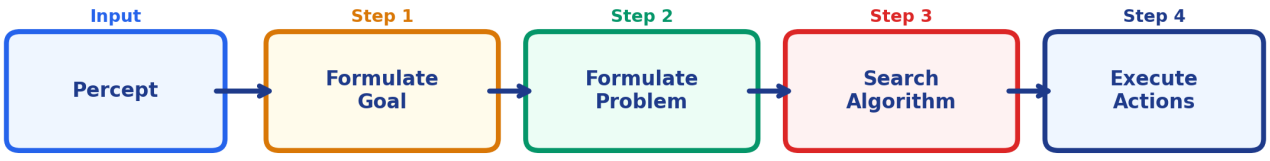


Figure 2.1: Operational Sequence of a Problem-Solving Agent

1.2 Formal Five-Component Single-State Problem Formulation

A well-defined classical search problem is formally defined by five components:

Component	Formal Definition	Example (Romania Driving / 8-Puzzle)
1. Initial State	The starting configuration of the agent in the environment.	In(Arad) or initial 3x3 tile grid layout.
2. Actions S(x)	Set of legal applicable actions available to the agent in state x.	Actions(Arad) = {Go(Zerind), Go(Sibiu), Go(Timisoara)}.
3. Transition Model	Result(s, a) function returning the state that results from doing action a in state s.	Result(In(Arad), Go(Zerind)) = In(Zerind).
4. Goal Test	Boolean predicate checking if state s is a goal state (explicit or implicit).	Explicit: s == In(Bucharest). Implicit: Checkmate(s).
5. Path Cost Function g(n)	Numeric cost function assigning a non-negative cost to a path, usually additive sum of step costs c(s, a, s').	Distance in miles between cities or depth count in 8-puzzle.

1.3 State Space vs Search Tree, Tree Search vs Graph Search

The **State Space** represents all possible valid configurations of the world. A **Search Tree** represents all explicit paths starting from the initial state node at the root. In graph search, identical states can be reached via multiple paths (redundant paths / loops). Unchecked loops cause infinite search trees!

> **Key Distinction: Tree Search** explores paths without keeping track of previously visited states, making it susceptible to infinite loops on cyclic graphs. **Graph Search** augments Tree Search with an **Explored Set (CLOSED list)** that stores all expanded nodes, strictly preventing redundant expansion of previously visited states.

Figure 2.2: Graph Search Infrastructure (Frontier vs Explored Set)

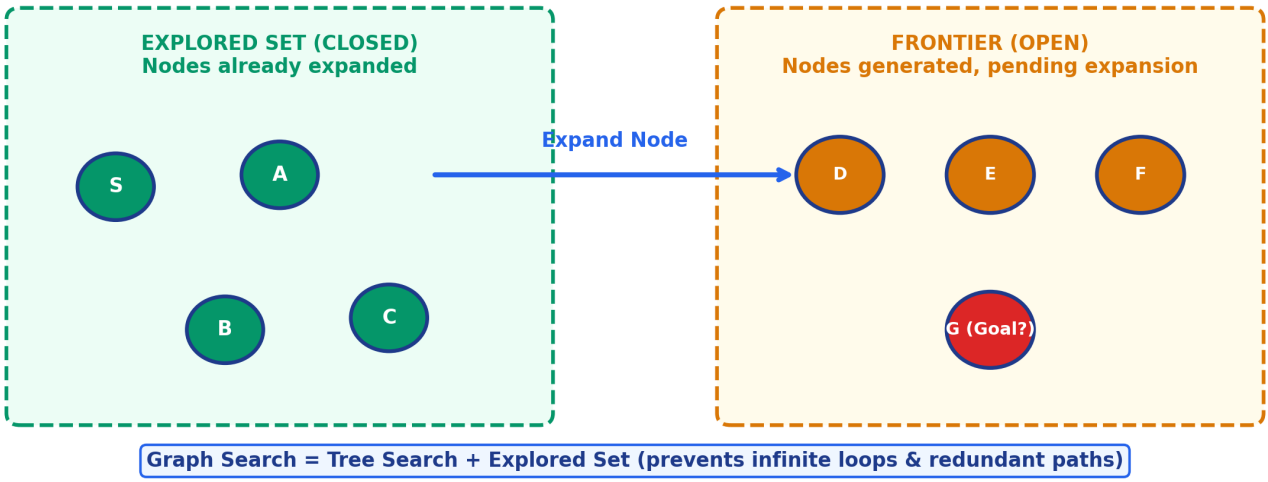


Figure 2.2: Graph Search Infrastructure (Frontier / OPEN vs Explored / CLOSED)

1.4 Search Performance Evaluation Criteria

Metric	Definition	Key Operational Formula / Parameters
Completeness	Is the algorithm guaranteed to find a solution if one exists?	Guaranteed for BFS and Graph Search in finite spaces.
Optimality	Does the strategy find the highest-quality (lowest path cost g) solution?	Requires unit step costs for BFS/IDS, or non-negative costs for UCS/A*.
Time Complexity	How many nodes are generated/expanded before reaching goal?	Expressed in terms of branching factor b and depth d: O(b^d).
Space Complexity	Maximum number of nodes stored in memory at any point?	Critical memory bottleneck: O(b^d) for BFS vs O(bm) for DFS.

2.0 UNINFORMED (BLIND) SEARCH STRATEGIES

Uninformed search algorithms operate solely on the problem definition (initial state, successor actions, goal test, path cost) without any domain-specific heuristic knowledge about how close a state is to the goal.

Figure 2.3: Uninformed Search Traversal Order Comparison

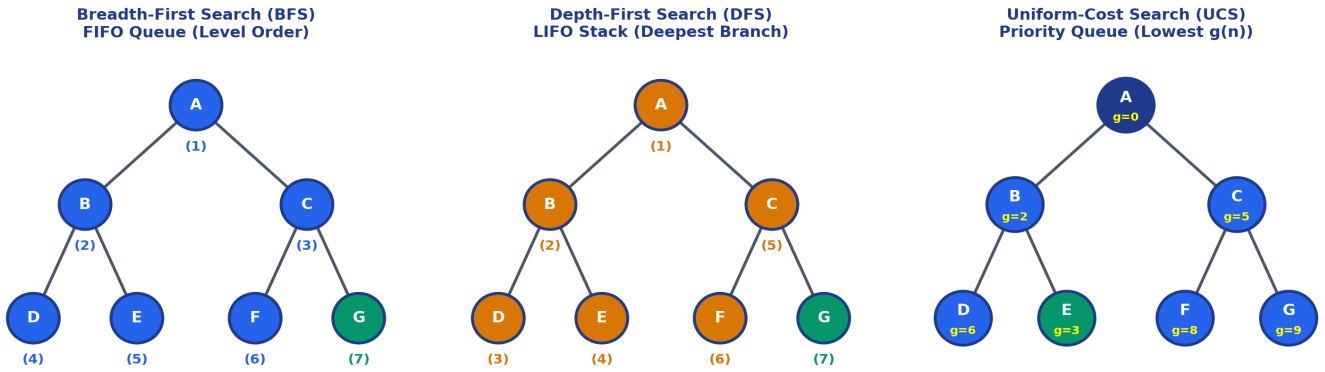


Figure 2.3: Comparison of Node Traversal Orders in Uninformed Search Strategies

2.1 Detailed Analysis of Uninformed Search Methods

Strategy	Data Structure	Time Complexity	Space Complexity	Complete?	Optimal?
Breadth-First (BFS)	FIFO Queue	O(b^(d+1))	O(b^(d+1)) [Exponential]	Yes (if b is finite)	Yes (unit step costs)
Depth-First (DFS)	LIFO Stack	O(b^m)	O(b * m) [Linear]	No (infinite paths)	No
Uniform-Cost (UCS)	Priority Queue [g(n)]	O(b^(1 + floor(C*/epsilon)))	O(b^(1 + floor(C*/epsilon)))	Yes (if cost >= epsilon)	Yes (general costs)
Depth-Limited (DLS)	LIFO Stack + Limit l	O(b^l)	O(b * l)	No (if l < d)	No
Iterative Deepening (IDS)	LIFO Stack (increasing l)	O(b^d)	O(b * d) [Linear]	Yes (if b is finite)	Yes (unit step costs)

Why Iterative Deepening Search (IDS) is Preferred: IDS combines the memory efficiency of DFS [O(bd)] with the completeness and optimality guarantees of BFS [O(b^d)]. It repeatedly runs Depth-Limited Search with increasing depth bounds l = 0, 1, 2, ..., d. Although it regenerates upper levels, the overhead is negligible because the majority of nodes lie at the bottom depth level d.

3.0 INFORMED (HEURISTIC) SEARCH STRATEGIES

Informed search strategies utilize domain-specific knowledge via a **Heuristic Function h(n)**, which estimates the remaining path cost from node n to the nearest goal state.

3.1 Mathematical Properties of Heuristic Functions

• **Admissibility:** A heuristic h(n) is admissible if it *never overestimates* the actual minimum cost h*(n) to reach the goal: **0 <= h(n) <= h*(n)**. Admissibility guarantees optimality in Tree-Search A*.

• **Consistency (Monotonicity):** A heuristic h(n) is consistent if for every node n and every successor n' generated by action a: **h(n) <= c(n, a, n') + h(n')**. Consistency ensures that f(n) values are non-decreasing along any path, guaranteeing optimality in Graph-Search A* without reopening closed nodes.

• **Dominance:** If h2(n) >= h1(n) for all n (and both are admissible), h2 *dominates* h1 and will expand fewer or equal nodes during A* search.

3.2 Greedy Best-First Search vs A* Search

Feature / Property	Greedy Best-First Search (GBFS)	A* Search
Evaluation Function f(n)	f(n) = h(n)	f(n) = g(n) + h(n)
Node Selection Focus	Purely greedy (selects lowest h(n))	Balances path cost g(n) + estimated remaining h(n)
Completeness	No (can get stuck in loops in graph search)	Yes (with admissible/consistent h)
Optimality	No (may select suboptimal path)	Yes (mathematically guaranteed shortest path)
Time Complexity	O(b^m) worst case	O(b^d) exponential (depends on heuristic quality)
Space Complexity	O(b^m) stores all nodes in memory	O(b^d) stores all frontier nodes

Figure 2.4: A* Informed Search Execution Tree showing f(n) = g(n) + h(n)

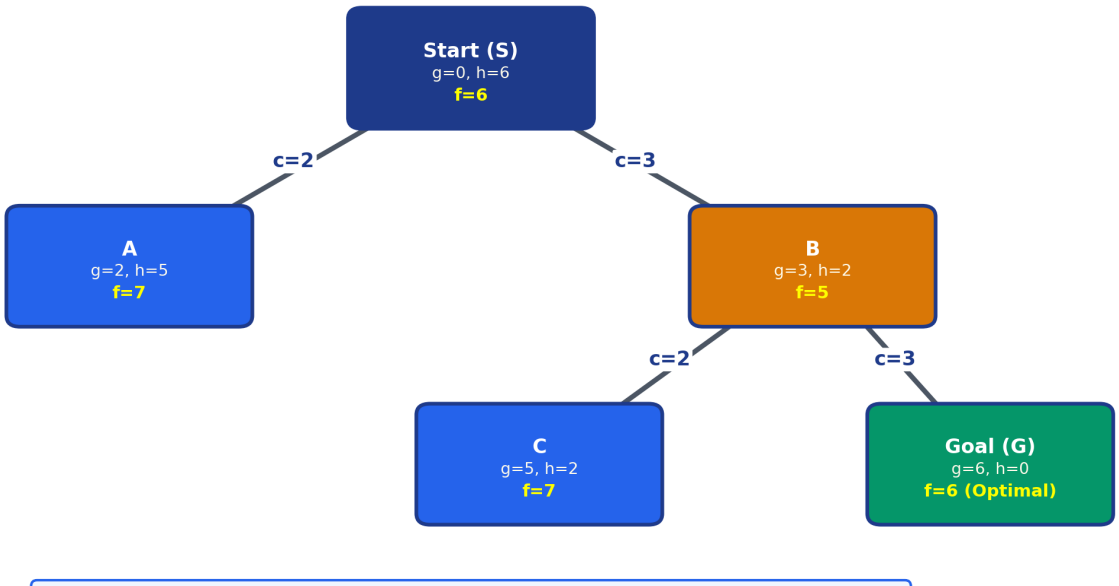


Figure 2.4: A* Search Tree Execution Showing Evaluation Function f(n) = g(n) + h(n)

3.3 Step-by-Step Solved 8-Puzzle Numerical Trace (A* Search)

8-Puzzle Problem Instance: Initial State [[2,8,3],[1,6,4],[7,0,5]], Goal State [[1,2,3],[8,0,4],[7,6,5]]. Using Misplaced Tiles Heuristic h(n).

Step / Expansion	Node (State ID)	Path Cost g(n)	Heuristic h(n)	Evaluation f(n)	OPEN List Status (Frontier)
Step 1	Start Node (S)	0	3 (Misplaced: 2,8,6)	f = 0 + 3 = 3	[S(f=3)]
Step 2	Expand S -> Up (A)	1	4	f = 1 + 4 = 5	[B(f=3), A(f=5), C(f=5)]
Step 2	Expand S -> Left (B)	1	2 (Misplaced: 6,8)	f = 1 + 2 = 3	**B selected next (min f)**
Step 2	Expand S -> Right (C)	1	4	f = 1 + 4 = 5	---
Step 3	Expand B -> Left (B1)	2	1 (Misplaced: 6)	f = 2 + 1 = 3	**B1 selected next (min f)**
Step 4	Expand B1 -> Down (Goal)	3	0 (Goal Reached!)	f = 3 + 0 = 3	**GOAL DISCOVERED! Path Length = 3**

4.0 LOCAL SEARCH ALGORITHMS & OPTIMIZATION

Local search algorithms operate on the **current state** rather than tracking execution paths. They are ideal for pure optimization problems (e.g., 8-Queens, TSP, VLSI design) where the path to the goal is irrelevant.

4.1 Hill Climbing Search & State-Space Landscapes

Hill Climbing (Steepest Ascent) continuously moves in the direction of increasing value (uphill). It terminates when it reaches a state where no neighbor has a higher value.

Figure 2.5: Hill-Climbing State-Space Landscape & Pathological Pitfalls

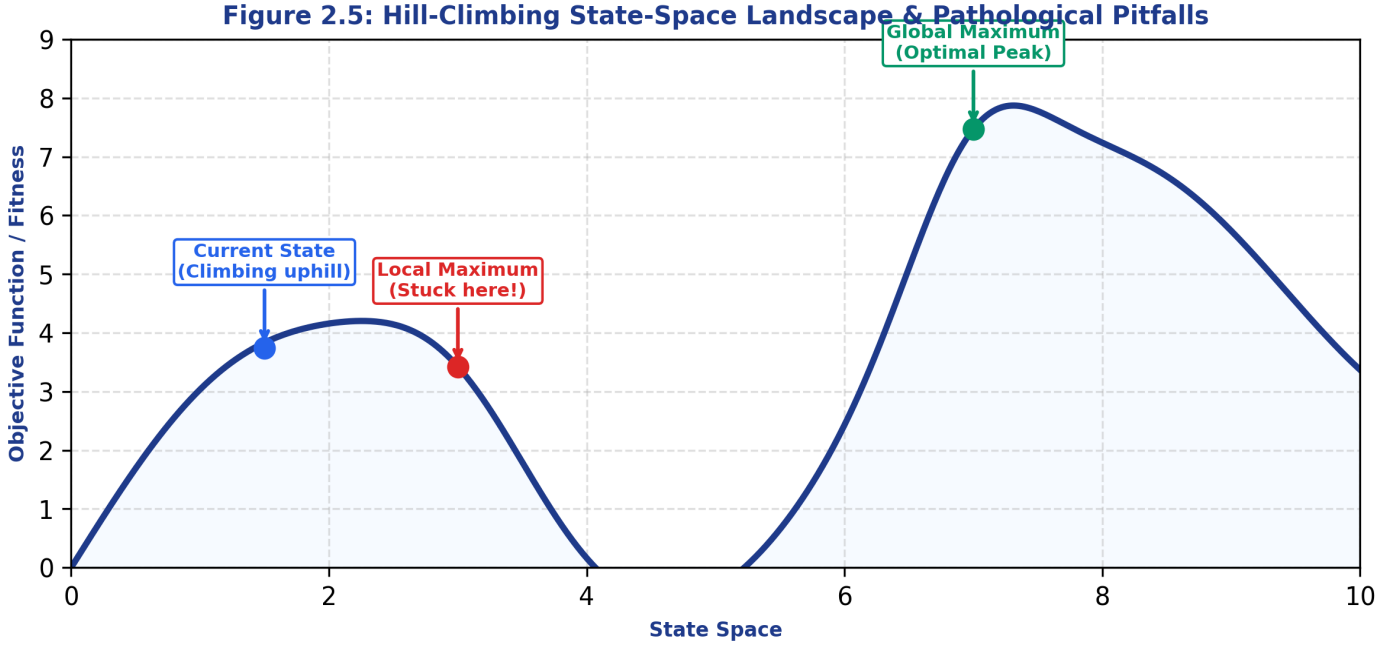


Figure 2.5: Hill-Climbing State-Space Landscape showing Local Maxima, Plateau, and Ridge

Pathological Pitfall	Landscape Description	Remedy / Algorithmic Variant
1. Local Maximum	A peak higher than all neighboring states, but lower than the global maximum.	**Random-Restart Hill Climbing:** Conducts multiple searches from randomly generated starting states.
2. Plateau / Shoulder	A flat area where all neighboring states have the exact same evaluation value.	**Allow Sideways Moves:** Execute limited consecutive flat steps, or use First-Choice Hill Climbing.
3. Ridge	A sequence of local maxima along a narrow slope where grid directions go downhill.	**Execute Multiple Rules Simultaneously:** Move diagonally or sample multiple search vectors.

4.2 Genetic Algorithms (GA)

A **Genetic Algorithm** is a population-based local search strategy inspired by natural biological evolution and natural selection.

Figure 2.6: Genetic Algorithm (GA) Evolutionary Cycle

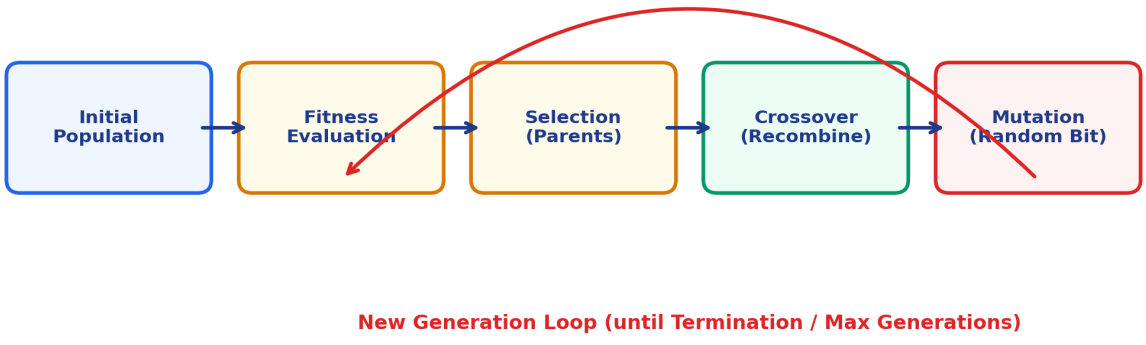


Figure 2.6: Evolutionary Process Flowchart of Genetic Algorithms

4.3 Worked Generation Example of Genetic Algorithm

Problem: Maximize f(x) = x^2 for 4-bit binary strings (x in [0, 15]).

String ID	Chromosome	x Value	Fitness f(x) = x^2	Selection Prob p_i	Actual Count in Mating Pool
P1	0110	6	36	36 / 293 = 12.3%	1
P2	1100	12	144	144 / 293 = 49.1%	2 (Selected twice!)
P3	0101	5	25	25 / 293 = 8.5%	0 (Discarded)
P4	1010	10	100	100 / 293 = 34.1%	1

Recombination: Crossover between P2 (1100) and P4 (1010) at position 2 -> Child 1: 1110 (x=14, f=196!). **Mutation:** Random bit-flip introduces new genetic diversity.

5.0 ADVERSARIAL SEARCH & GAME PLAYING

Adversarial search involves competitive environments where two or more optimal agents have conflicting goals (zero-sum games). In a 2-player zero-sum game, MAX seeks to maximize utility while MIN seeks to minimize it.

5.1 Minimax Algorithm & Game Trees

Figure 2.7: Solved Minimax Game Tree with Backed-Up Values

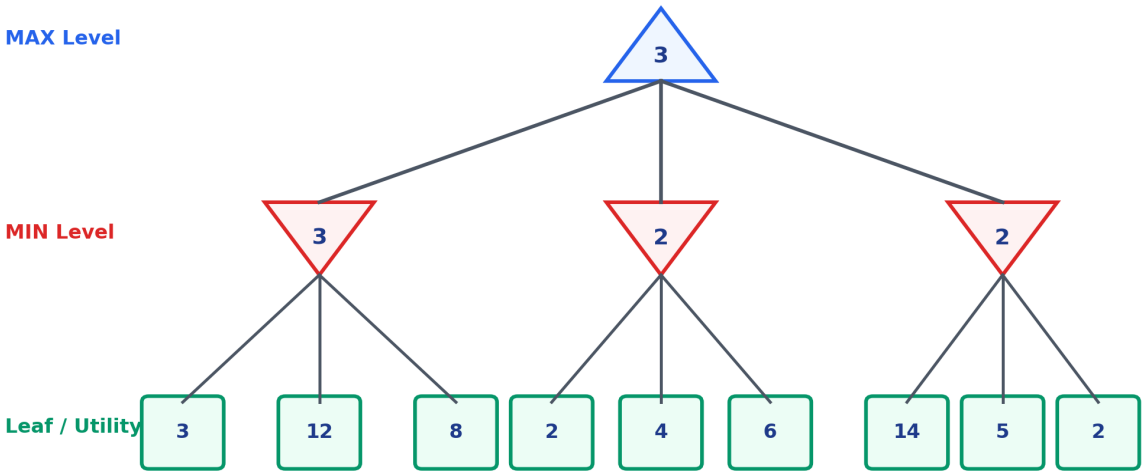


Figure 2.7: Solved Minimax Game Tree with Backed-Up Values

5.2 Alpha-Beta Pruning Algorithm

Alpha-Beta Pruning optimizes Minimax by eliminating subtrees that cannot possibly influence the final decision at the root node. It maintains two bounds along the search path:

- **Alpha (alpha):** The value of the best (highest-value) choice found so far along the path for **MAX**. Initialized to -infinity.
- **Beta (beta):** The value of the best (lowest-value) choice found so far along the path for **MIN**. Initialized to +infinity.
- **Alpha-Cutoff:** Given a MIN node, prune search below it if beta <= alpha.

• **Beta-Cutoff:** Given a MAX node, prune search below it if alpha >= beta.

Figure 2.8: Alpha-Beta Pruning Game Tree with Alpha-Cutoff and Pruned Subtrees

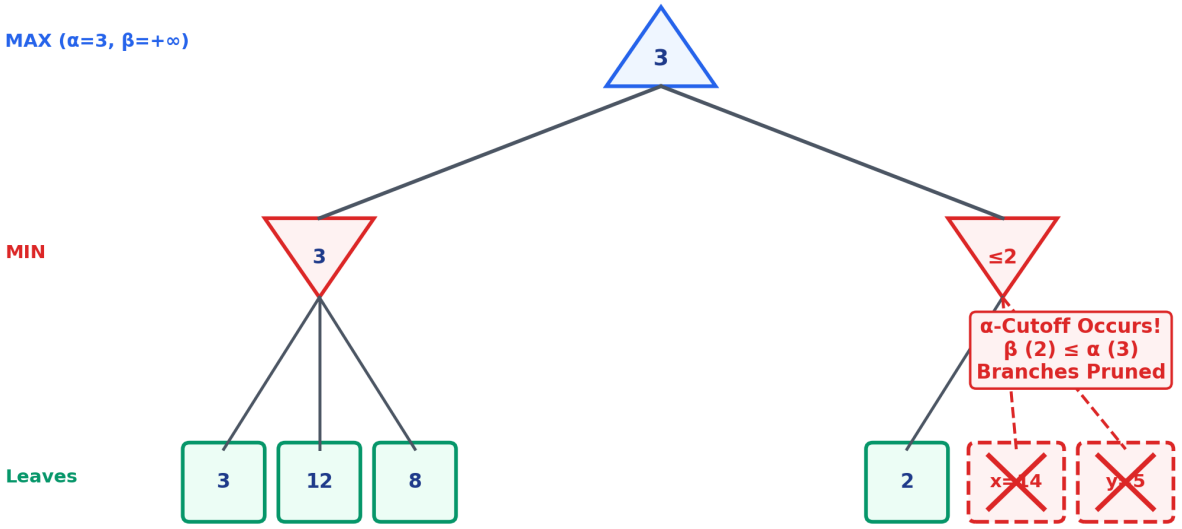


Figure 2.8: Alpha-Beta Pruning Game Tree showing Alpha-Cutoff and Pruned Subtrees

Parameter / Metric	Standard Minimax	Alpha-Beta Pruning (Optimal Move Ordering)
Time Complexity	O(b^m)	O(b^(m/2)) [Doubles search depth!]
Space Complexity	O(b * m)	O(b * m) [Depth-first storage]
Final Root Decision	Computes exact minimax decision	Computes EXACT same decision as Minimax
Search Tree Depth	Depth d	Effective Depth 2d for same compute time!

6.0 EXAM REVIEW, SOLVED PYQ BANK & SYLLABUS CHECKLIST

6.1 Frequently Confused Concepts & Common Exam Mistakes

Concept Pair	Common Student Misconception	Correct Technical Distinction
A* Tree vs Graph Search	Assuming admissible h guarantees optimality in Graph Search.	Admissibility is sufficient for Tree Search; **Consistency** is required for Graph Search.
BFS vs UCS Optimality	Believing BFS is always optimal for any edge cost.	BFS is optimal ONLY when all step costs are equal (unit cost). UCS is optimal for general non-negative costs.
Minimax Output	Thinking Alpha-Beta Pruning produces an approximate answer.	Alpha-Beta Pruning produces the **EXACT same minimax decision** as full Minimax.

6.2 Model Answers for Solved Probable Examination Questions

Q1 [2 Marks]: Define Admissible Heuristic and Consistent Heuristic.
Answer: An *Admissible Heuristic* h(n) never overestimates the true minimum cost to reach the goal: 0 <= h(n) <= h*(n). A *Consistent (Monotone) Heuristic* satisfies h(n) <= c(n, a, n') + h(n') for every node n and successor n'. Every consistent heuristic is admissible.

Q2 [5 Marks]: Differentiate between Uninformed and Informed Search. Compare BFS and A*.
Answer: Uninformed search uses only problem definitions (initial state, actions, goal test, cost) without heuristic guidance. Informed search uses domain knowledge via a heuristic h(n) estimating distance to goal.
BFS vs A* Comparison:
• Evaluation: BFS expands shallowest node first; A* expands node with min f(n) = g(n) + h(n).
• Optimality: BFS optimal only for unit step costs; A* optimal for general non-negative costs.
• Memory: Both have exponential space complexity O(b^d), but A* expands far fewer nodes in practice.

Q3 [10 Marks]: Explain Alpha-Beta Pruning with a neat diagram and trace table.
Answer Overview:
1. **Definition:** Alpha-Beta Pruning is an optimized depth-first search technique for 2-player zero-sum game trees that prunes provably irrelevant subtrees.
2. **Parameters:** alpha = MAX's best guaranteed choice (-infinity init); beta = MIN's best guaranteed choice (+infinity init).
3. **Cutoff Rules:** Prune at MIN node if beta <= alpha (Alpha-Cutoff); Prune at MAX node if alpha >= beta (Beta-Cutoff).
4. **Efficiency:** With optimal move ordering, time complexity reduces from O(b^m) to O(b^(m/2)), doubling search depth!

6.3 Unit 2 Master Syllabus Completeness Checklist

Syllabus Topic Area	Status	Verified Coverage in Notes
1. Problem-Solving Agents & Problem Formulation	COMPLETE	5-component single-state formulation, state space, search trees.
2. Uninformed Search Strategies	COMPLETE	BFS, DFS, UCS, DLS, IDS with complete complexity & trace table.
3. Informed (Heuristic) Search Strategies	COMPLETE	Heuristic properties (admissibility, consistency), GBFS, A* with 8-puzzle numerical.
4. Local Search & Optimization	COMPLETE	Hill Climbing, pitfalls (local max, plateau, ridge), Genetic Algorithm flowchart & numerical.
5. Adversarial Search & Game Playing	COMPLETE	Minimax algorithm, Alpha-Beta pruning diagram, alpha/beta cutoffs, move ordering.